



Verm (from Vermilingua) Entrega Final

1. Integrantes

Nombre	Apellido	Legajo	Email
Uriel	Sosa Vásquez	63250	usosavazquez@itba.edu.ar
Felipe	Venturino	63161	fventurino@itba.edu.ar

2. Repositorio

<https://github.com/urielsv/Verm>

3. Introducción

3.1 Motivación y Contexto

El análisis de tráfico de red es una tarea fundamental para múltiples áreas como la ciberseguridad, el monitoreo de redes, el monitoreo de redes, entre otros. Las herramientas tradicionales como Wireshark permiten capturar paquetes, pero no ofrecen un lenguaje específico de alto nivel que facilite la detección declarativa de patrones o alertas personalizadas sin necesidad de programar. En este contexto, surge la necesidad de un lenguaje de dominio específico (DSL) que permita definir filtros complejos, alertas condicionales y operaciones de agregación sobre paquetes de red capturados, con una sintaxis clara, expresiva y orientada a usuarios técnicos, pero no necesariamente expertos en programación de bajo nivel.

3.2 Propuesta de solución

Verm es un compilador diseñado para procesar un lenguaje DSL destinado al filtrado y análisis de tráfico de red. Su objetivo principal es brindar una interfaz declarativa que permita:

- Declarar variables y expresiones numéricas.
- Definir filtros por campos de los paquetes (como `ip.src`, `http.status`, etc.).
- Construir condiciones lógicas complejas.
- Aplicar funciones de agregación como `count`, `sum`, `avg`, `min` y `max`.
- Declarar alertas con condiciones y definir acciones a ejecutar

4. Modelo Computacional

4.1 Lenguaje

El lenguaje de Verm está diseñado como un DSL declarativo, con un dominio restringido al análisis de tráfico de red. Está orientado a definir reglas y condiciones de forma expresiva para identificar patrones dentro de archivos `.pcap`. A diferencia de otros lenguajes de propósito general, Verm reduce la complejidad semántica al limitarse a expresiones numéricas, lógicas y funciones agregadas sobre campos de paquetes de red.

La estructura del lenguaje permite la combinación de filtros básicos, condiciones compuestas, uso de variables, bloques de alertas con múltiples condiciones, y acciones asociadas.

4.2 Estructura del lenguaje

Elemento del lenguaje	Descripción
<code>int a = 4 + 1;</code>	Declaración de una variable entera
<code>filter ip.src == 192.168.1.1</code>	Filtro simple por campo de paquete
<code>alert when { condition } send to "user"</code>	alerta con condición y acción asociada
<code>count(http.status >= 500) > 3</code>	Uso de función de agregación en condición

4.3 Principios de Composición

El lenguaje de Verm se construye sobre principios de composición claros:

1. **Bloques estructurados:** las alertas y condiciones se agrupan mediante llaves, lo cual permite una representación jerárquica y clara de flujo lógico.
2. **Compatibilidad con variables:** las variables permiten factorizar expresiones comunes y mejorar la claridad semántica del código.
3. Separación de cláusulas: cada unidad del lenguaje (filter, alert, etc) tiene una sintaxis definida y delimitada, evitando ambigüedades gramaticales.

5. Implementación

5.1 Frontend

5.1.1 Analizador Léxico

El analizador léxico de Verm fue implementado utilizando **Flex**, y define múltiples **start conditions** (%x) para manejar distintos contextos, como comentarios multilinea (MULTILINE_COMMENT), literales de strings (STRING_LITERAL) y declaraciones de variables (VAR_IDENTIFIER). Esta separación permitió un control fino del análisis, habilitando reglas específicas según el tipo de contenido que se está procesando.

Las **palabras clave** del lenguaje (como alert, filter, group, count, sum, timespan, etc.) se reconocen con reglas específicas que llaman a funciones como KeywordLexemeAction(...), lo que encapsula la lógica de construcción del token y el registro de ubicación del lexema para debugging posterior.

Ejemplo: "count" { return KeywordLexemeAction(createLexicalAnalyzerContext(), COUNT); }

Los **identificadores de variables** tras la palabra clave int son tratados con una transición de contexto hacia VAR_IDENTIFIER, lo que garantiza una detección inmediata del nombre de la variable tras la declaración:

"int" { BEGIN(VAR_IDENTIFIER); return TypeLexemeAction(createLexicalAnalyzerContext(), INT_TYPE); }

```

<VAR_IDENTIFIER>{identifier} {

    BEGIN(INITIAL);

    return IdentifierLexemeAction(createLexicalAnalyzerContext());

}

```

5.2.2 Analizador Sintáctico

El análisis sintáctico de Verm fue implementado utilizando **Bison**, definiendo una gramática LL(1) que traduce el código fuente en una estructura jerárquica de nodos del AST (Abstract Syntax Tree). Cada nodo corresponde a una construcción del lenguaje como alert, filter, define, expression, etc., y se conecta mediante acciones semánticas escritas en C.

Tipos y Semántica

Se definió una **unión de tipos** (%union) que incluye tanto tipos primitivos (enteros, strings) como punteros a estructuras del AST (Expression*, Condition*, Statement*, etc.), permitiendo mapear cada no terminal a su representación en memoria:

```

...

%union {

    char* string;

    int integer;

    Token token;

    Constant* constant;

    Expression* expression;

    Statement* statement;

    Condition* condition;

    ...

}

...

```

Cada símbolo no terminal está tipado con %% %type <...> y se conectan directamente a las acciones semánticas. Por ejemplo:

expression:

```

expression ADD expression { $$ = ArithmeticExpressionSemanticAction($1, $3, ADDITION); }

```

El programa es una secuencia de **statements**, entre los que se incluyen declaraciones (`int a = 5;`), filtros, capturas, extracciones y alertas. Por ejemplo:

alert_statement:

```
ALERT WHEN condition SEND TO STRING SEMICOLON {  
    $$ = AlertStatementSemanticAction($3, $6);  
};
```

Las expresiones matemáticas están completamente soportadas, con reglas de precedencia (`%left`, `%right`) para los operadores aritméticos y lógicos (`+`, `-`, `*`, `/`, `and`, `or`, etc.).

El lenguaje también tiene soporte completo para agregaciones como `count`, `sum`, `avg`, tanto en condiciones como expresiones. Estas se pueden aplicar sobre campos o expresiones completas y son tratadas uniformemente por la gramática:

aggregation_pattern:

```
COUNT OPEN_PARENTHESIS field CLOSE_PARENTHESIS comparison_operator expression {  
    $$ = AggregationPatternConditionSemanticAction($3, $5, $6, AGGREGATION_COUNT);  
}
```

Además se permite `count(*)` para conteos globales.

Finalmente cada tipo no terminal define su **destructor personalizado** (`%destructor`) para liberar correctamente la memoria asignada al AST, evitando leaks durante la compilación:

```
%destructor { releaseExpression($$); } <expression>
```

```
%destructor { releaseCondition($$); } <condition>
```

```
%destructor { free($$); } <string>
```

5.2 Backend

El backend de Verm está estructurado en cuatro módulos principales que implementan el procesamiento semántico, la generación de código, el análisis de dominio específico y el runtime de ejecución.

5.2.1 Análisis Semántico

El módulo de análisis semántico (`semantic-analysis/`) valida la corrección semántica del AST generado por el frontend. Está compuesto por:

- Sistema de Tipos (`TypeSystem.h/c`): Define los tipos de datos soportados por el lenguaje, incluyendo tipos primitivos (enteros, strings) y tipos específicos del dominio como campos de paquetes de red (IP, puertos, protocolos HTTP).
- Tabla de Símbolos (`SymbolTable.h/c`): Implementa un sistema de gestión de ámbitos (scopes) que permite:

- Declarar variables en diferentes niveles de anidamiento
- Resolver referencias a símbolos según las reglas de visibilidad
- Validar que las variables estén declaradas antes de su uso
- Gestionar el ciclo de vida de los símbolos durante la compilación

Analizador Semántico (`SemanticAnalyzer.h/c`): Coordina la validación semántica completa del programa:

- Verifica la consistencia de tipos en expresiones y asignaciones
- Valida que los campos de protocolo referenciados existan y sean accesibles

- Comprueba la sintaxis de filtros BPF generados
- Asegura que las funciones de agregación se apliquen correctamente

5.2.3 Análisis de Dominio Específico

El módulo de dominio específico (domain-specific/) implementa funcionalidades especializadas para el análisis de tráfico de red: Calculadora (Calculator.h/c): Proporciona funciones matemáticas y estadísticas:

- Operaciones aritméticas básicas con manejo de errores
- Cálculo de estadísticas descriptivas (media, mediana, desviación estándar)
- Detección de patrones anómalos en el tráfico
- Análisis de distribución de protocolos y puertos

Detección de Anomalías: Implementa algoritmos especializados para:

- Detección de port scanning mediante análisis de patrones de conexión
- Identificación de ataques DDoS basados en volumen de tráfico
- Detección de ataques HTTP lentos (slow HTTP attacks)
- Análisis de latencia de conexiones TCP
- Detección de exfiltración de datos mediante análisis de tamaños de paquetes

Perfiles de Comportamiento: Mantiene estructuras de datos para:

- Perfiles de hosts con métricas de actividad
- Perfiles de conexiones con análisis de latencia
- Perfiles de endpoints HTTP con estadísticas de respuesta

5.2.4 Runtime de Ejecución

El módulo de runtime (runtime/) proporciona la infraestructura para procesar archivos PCAP y generar salidas: Runtime PCAP (PcapRuntime.h/c): Maneja la interacción con archivos de captura:

- Inicialización y gestión de contextos PCAP
- Compilación de filtros BPF para optimizar el procesamiento
- Callbacks para procesamiento de paquetes individuales
- Extracción de metadatos de paquetes (timestamps, longitudes)

Extractores de Protocolos (ProtocolExtractors.h/c): Implementa parsers especializados para:

- Capa IP: Extracción de direcciones fuente/destino, TTL, protocolo
- Capa TCP: Puertos, números de secuencia, flags, ventana
- Capa UDP: Puertos, longitud de datagrama
- Capa HTTP: Métodos, URLs, códigos de estado, headers, cookies

Generadores de Salida:

- HTML (HtmlOutput.h/c): Crea dashboards web completos con Bootstrap y Chart.js
- CSV (CsvOutput.h/c): Genera reportes en formato tabular para análisis posterior (no se terminó)

5.2.5 Flujo de Ejecución

El backend sigue un flujo secuencial bien definido:

1. Validación Semántica: El AST se valida contra reglas semánticas y se construye la tabla de símbolos
2. Configuración de Captura: Se identifican declaraciones de import/capture para determinar el archivo PCAP
3. Procesamiento de Paquetes: Se itera sobre cada paquete aplicando filtros y extrayendo campos relevantes
4. Evaluación de Condiciones: Se evalúan las condiciones del DSL contra los datos extraídos
5. Agregación de Resultados: Se aplican funciones de agregación y agrupación según las consultas
6. Generación de Reportes: Se crean visualizaciones HTML con métricas, tablas y gráficos
7. Detección de Anomalías: Se ejecutan algoritmos de detección de patrones sospechosos

Esta arquitectura modular permite una separación clara de responsabilidades, facilitando el mantenimiento y la extensión del sistema para nuevos protocolos o tipos de análisis.

7. Conclusiones

7.1 Logros y Contribuciones

El proyecto Verm representa un esfuerzo significativo en el desarrollo de un lenguaje de dominio específico para el análisis de tráfico de red. Se buscó implementar:

- Un compilador completo con frontend (lexer y parser) utilizando Flex y Bison
- Un sistema de tipos robusto que valida la corrección semántica de las consultas
- Una arquitectura modular que separa claramente las responsabilidades entre análisis léxico, sintáctico, semántico y generación de código
- Extractores de protocolos especializados para IP, TCP, UDP y HTTP
- Un sistema de generación de reportes HTML con visualizaciones interactivas
- Algoritmos de detección de anomalías para identificar patrones de seguridad

7.2 Desafíos Técnicos Enfrentados

Durante el desarrollo se encontraron varios desafíos significativos: Complejidad del Manejo de Memoria: La gestión de memoria en C, especialmente en el procesamiento de paquetes PCAP, resultó

ser particularmente compleja. Los segment faults (segfaults) experimentados indican problemas en la gestión de punteros y la liberación de memoria dinámica.

Integración de Componentes: La comunicación entre el frontend (AST) y el backend (runtime) presentó dificultades, especialmente en el paso de información contextual durante la ejecución. Los errores de runtime sugieren que la información no se está propagando correctamente entre las diferentes capas del sistema.

Procesamiento de Datos Binarios: El análisis de archivos PCAP requiere un manejo cuidadoso de estructuras de datos binarias y offsets de memoria, lo cual es propenso a errores cuando no se maneja con precisión.

7.3 Limitaciones Identificadas

Problemas de Estabilidad: Los errores de segmentación fault indican que el sistema no está completamente estable para el procesamiento de archivos PCAP reales. Esto sugiere la necesidad de:

- Mejorar la validación de entrada en el runtime
- Implementar un manejo más robusto de errores
- Revisar la gestión de memoria en el procesamiento de paquetes
- Añadir verificaciones de límites más estrictas

Cobertura de Protocolos: Aunque se implementaron extractores para los protocolos principales (IP, TCP, UDP, HTTP), el sistema podría beneficiarse de soporte para protocolos adicionales como DNS, DHCP, o protocolos de aplicación específicos.

Optimización de Rendimiento: El procesamiento secuencial de paquetes podría optimizarse para archivos PCAP de gran tamaño mediante técnicas de paralelización o procesamiento por lotes.

7.4 Aprendizajes y Reflexiones

Este proyecto nos proporcionó una experiencia valiosa en el desarrollo de compiladores y en el entendimiento del procesamiento de datos de red. Algunos de los puntos fueron:

- La importancia de una arquitectura bien diseñada desde las primeras etapas
- La complejidad inherente al manejo de memoria en sistemas de bajo nivel
- La necesidad de buenos tests en cada capa del compilador
- Evaluación temprana para las necesidades de un DSL

7.5 Trabajo Futuro

Para completar y mejorar el sistema, se buscaría:

1. Debugging y Corrección de Errores: Resolver los problemas de segmentación fault mediante análisis detallado del flujo de datos y la gestión de memoria, implementación de profiling para la mejora de detección de problemas y performance.
2. Sistema de Pruebas: Implementar un conjunto completo de pruebas unitarias y de integración

3. Documentación: Mejorar la documentación del código y crear guías de usuario para que no requiera de instalación de bibliotecas externas.
4. Optimizaciones: Implementar optimizaciones de rendimiento y manejo de archivos grandes
5. Extensiones: Añadir soporte para protocolos adicionales y nuevas funcionalidades de análisis
6. Nuevas formas más detalladas de producir salida, con mayor manejabilidad

A pesar de los desafíos técnicos encontrados, el proyecto Verm demuestra el potencial de los lenguajes de dominio específico para simplificar tareas complejas de análisis de red, proporcionando una base sólida para futuras mejoras y extensiones.